

✓ EDA Notebook

✓ 0. Setup Environment

✓ 0.a Install Mandatory Packages

Do not modify this code before running it

```
# Do not modify this code
import os
import sys
from pathlib import Path

COURSE = "36106"
ASSIGNMENT = "AT3"
DATA = "data"

asgmt_path = f"{COURSE}/assignment/{ASSIGNMENT}"
root_path = "./"

#print("##### Install required Python packages #####")
! pip install -q -r https://raw.githubusercontent.com/aso-uts/labs\_datasets/main/36106-mlaa/requirements.txt

if os.getenv("COLAB_RELEASE_TAG"):

    from google.colab import drive
    from pathlib import Path

    #print("\n##### Connect to personal Google Drive #####")
    gdrive_path = "/content/gdrive"
    drive.mount(gdrive_path)
    root_path = f"{gdrive_path}/MyDrive/"

print("\n##### Setting up folders #####")
folder_path = Path(f"{root_path}/{asgmt_path}/") / DATA
folder_path.mkdir(parents=True, exist_ok=True)
print(f"\nYou can now save your data files in: {folder_path}")

if os.getenv("COLAB_RELEASE_TAG"):
    %cd {folder_path}

from IPython.display import display, HTML

def print_tile(key, value, size="h1"):
    return display(HTML(f""<p style="color:grey">{key}</p><{size} font-size: 3em>{value}</{size}>""))

🔗 Mounted at /content/gdrive

##### Setting up folders #####

You can now save your data files in: /content/gdrive/MyDrive/36106/assignment/AT3/data
/content/gdrive/MyDrive/36106/assignment/AT3/data
```

✓ 0.b Disable Warnings Messages

Do not modify this code before running it

```
# Do not modify this code
import warnings
warnings.simplefilter(action='ignore', category=FutureWarning)
```

✓ 0.c Install Additional Packages

If you are using additional packages, you need to install them here using the command: ! pip install <package_name>

```
# <Student to fill this section>
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from datetime import timedelta
import pickle
from IPython.display import display

import missingno as msno
```

0.d Import Packages

```
# <Student to fill this section>
import pandas as pd
import altair as alt
```

A. Project Description

```
# <Student to fill this section>
group_name = "Group 18"
student_name = "Vonny Yudianto"
student_id = "25741719"
```

```
# Do not modify this code
print_tile(size="h1", key='group_name', value=group_name)
```

↔ group_name

Group 18

```
# Do not modify this code
print_tile(size="h1", key='student_name', value=student_name)
```

↔ student_name

Vonny Yudianto

```
# Do not modify this code
print_tile(size="h1", key='student_id', value=student_id)
```

↔ student_id

25741719

B. Data Understanding

B.1 Load Dataset

```
# Do not modify this code
try:
    df = pd.read_csv(folder_path / "customer_details.csv")
except Exception as e:
    print(e)
```

```
try:
    df1 = pd.read_csv(folder_path / "customer_usage.csv")
except Exception as e:
    print(e)
```

B.2 Explore Dataset

You can add more cells in this section

```
# <Student to fill this section>
df.head()
```

	Unnamed: 0	customer_id	full_name	gender	dob	age	is_young_adult	is_senior	birth_country	passport_number	...	city	postcode
0	0	9acb3a1e-a040-4280-887b-ef33381c7a16	Alexa Chung	Female	1974-10-10	50	No	No	AU	131053534	...	NaN	NaN
1	1	45360147-4c3e-4f3d-8d8d-a6aa8f0bb47f	Mrs. Megan	Female	1981-09-27	43	No	No	AU	379521345	...	NaN	NaN
2	2	8345bdc5-75fd-41da-8ce5-7828e5a27666	Amy Rose	Female	1989-09-16	35	No	No	PH	794022868	...	NaN	NaN
3	3	0ec226b1-22e6-4b98-919f-c1c0cd183ea7	Ryan Peterson	Male	2001-02-03	24	Yes	No	AU	M21113240	...	NaN	NaN
4	4	f588a5e1-7b36-4db2-af9a-2c14e041fcdb	Shelby Meyer	Female	1977-02-11	48	No	No	AU	956682468	...	NaN	NaN

5 rows × 36 columns

```
df.tail()
```

	Unnamed: 0	customer_id	full_name	gender	dob	age	is_young_adult	is_senior	birth_country	passport_number	...	city	p
11683	11683	e9f08d32-da2e-4f90-b4d5-0a211489ad95	Tammy Johnson	Female	1959-07-30	65	No	Yes	AU	626987514	...	NaN	
11684	11684	b34d96de-86a9-42c2-bafd-fe943d6b33f5	Kevin Carr	Male	1992-05-06	32	No	No	AU	F33921196	...	NaN	
11685	11685	4f7237ff-9a24-491f-9f7c-8b7ed2fd0f42	Gregory Hunt	Male	1994-07-15	30	No	No	AU	U21245458	...	East Rachelfort	
11686	11686	f44e11d5-d0bf-442c-b7e3-199a2a31b354	James Rogers	Male	2002-10-28	22	Yes	No	AU	E12618421	...	NaN	
11687	11687	e4eb2526-48b8-4431-b0cc-a4cfd7e8f9b	Ryan Green	Male	1957-12-30	67	No	Yes	AU	709720306	...	NaN	

5 rows × 36 columns

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 11688 entries, 0 to 11687
Data columns (total 36 columns):
#   Column                Non-Null Count  Dtype

```

```

---
0  Unnamed: 0      11688 non-null int64
1  customer_id     11688 non-null object
2  full_name       11688 non-null object
3  gender          11688 non-null object
4  dob            11688 non-null object
5  age            11688 non-null int64
6  is_young_adult  11688 non-null object
7  is_senior       11688 non-null object
8  birth_country   11688 non-null object
9  passport_number 11688 non-null object
10 passport_expiry 11688 non-null object
11 phone_number    11688 non-null object
12 email           11688 non-null object
13 marital_status  11688 non-null object
14 has_children    11688 non-null object
15 number_dependents 11688 non-null int64
16 Gender          1688 non-null object
17 profile         1688 non-null object
18 locale          1688 non-null object
19 prefix          1116 non-null object
20 first_name      1688 non-null object
21 last_name       1688 non-null object
22 secondary_address 1688 non-null object
23 building_number 1688 non-null float64
24 street_name     1688 non-null object
25 street_suffix   1688 non-null object
26 city            1688 non-null object
27 postcode        1688 non-null float64
28 state_abbr      1688 non-null object
29 address         1688 non-null object
30 cc_number       1688 non-null float64
31 cc_expiry       1688 non-null object
32 cc_security_code 1688 non-null float64
33 company_name    1688 non-null object
34 company_suffix  1688 non-null object
35 user_agent      1688 non-null object
dtypes: float64(4), int64(3), object(29)
memory usage: 3.2+ MB

```

```
# Display unique items from each column
```

```

def display_unique(df):
    listUniqueItem = []
    for col in df.columns :
        listUniqueItem.append( [col, df[col].nunique(), df[col].isna().sum(), df[col].unique()])

    tableDesc = pd.DataFrame(columns=['Column Name', 'Total of Unique', 'null', 'Unique Sample'],
                              data=listUniqueItem)

    return tableDesc

display_unique(df)

```

	Column Name	Total of Unique	null	Unique Sample
0	Unnamed: 0	11688	0	[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13,...
1	customer_id	8731	0	[9acb3a1e-a040-4280-887b-ef33381c7a16, 4536014...
2	full_name	8118	0	[Alexa Chung, Mrs. Megan, Amy Rose, Ryan Peter...
3	gender	3	0	[Female, Male, Unknown]
4	dob	7282	0	[1974-10-10, 1981-09-27, 1989-09-16, 2001-02-0...
5	age	75	0	[50, 43, 35, 24, 48, 63, 36, 57, 19, 38, 49, 6...
6	is_young_adult	2	0	[No, Yes]
7	is_senior	2	0	[No, Yes]
8	birth_country	13	0	[AU, PH, IN, CO, US, NZ, BR, CA, ZA, FR, TH, I...
9	passport_number	8252	0	[131053534, 379521345, 794022868, M21113240, 9...
10	passport_expiry	1751	0	[2029-11-21, 2028-04-11, 2028-12-13, 2027-05-2...
11	phone_number	8224	0	[2948.0872, 02-2279-7525, (02).1955.3942, 03-6...
12	email	8045	0	[jamespeterson@example.com, karenhester@exampl...
13	marital_status	2	0	[Yes, No]
14	has_children	2	0	[Yes, No]
15	number_dependents	10	0	[2, 0, 3, 1, 4, 5, 9, 6, 7, 8]
16	Gender	3	10000	[nan, Female, Male, Unknown]
17	profile	1688	10000	[nan, {"locale": "en_AU", "prefix": "Dr.", "fi...
18	locale	13	10000	[nan, en_AU, en_IN, fil_PH, en_NZ, en_US, en_P...
19	prefix	5	10572	[nan, Dr., Mrs., Mr., Ms., Miss]
20	first_name	517	10000	[nan, Angel, Lori, Christopher, Craig, Todd, H...
21	last_name	745	10000	[nan, Farley, Freeman, Tate, Steele, Lucas, Mo...
22	secondary_address	828	10000	[nan, Level 1 , Apt. 541 , Apt. 376 , Suite 91...
23	building_number	472	10000	[nan, 57.0, 42.0, 463.0, 3.0, 342.0, 52.0, 546...
24	street_name	1667	10000	[nan, Barr Highway, Lopez Common, Patterson Ro...
25	street_suffix	200	10000	[nan, Gap, Colonnade, Lower, Cross, Riverway, ...
26	city	1572	10000	[nan, Durhampart, Campbellborough, Aliciaport,...
27	postcode	894	10000	[nan, 2626.0, 2547.0, 2687.0, 2021.0, 968.0, 5...
28	state_abbr	8	10000	[nan, VIC, QLD, TAS, WA, ACT, NT, SA, ...
29	address	1688	10000	[nan, Level 1 57 Barr Highway Gap, Durhampart ...
30	cc_number	1688	10000	[nan, 4746551385440.0, 213128791282074.0, 5851...
31	cc_expiry	121	10000	[nan, 11/34, 04/26, 07/25, 04/31, 06/26, 11/33...
32	cc_security_code	920	10000	[nan, 295.0, 559.0, 3612.0, 4237.0, 392.0, 709...
33	company_name	1622	10000	[nan, Butler Inc, Graham, Estrada and Ford, Co...
34	company_suffix	6	10000	[nan, Group, Ltd, LLC, and Sons, Inc, PLC]

df.shape

(11688, 36)

df.columns

Index(['Unnamed: 0', 'customer_id', 'full_name', 'gender', 'dob', 'age', 'is_young_adult', 'is_senior', 'birth_country', 'passport_number', 'passport_expiry', 'phone_number', 'email', 'marital_status', 'has_children', 'number_dependents', 'Gender', 'profile', 'locale', 'prefix', 'first_name', 'last_name', 'secondary_address', 'building_number', 'street_name', 'street_suffix', 'city', 'postcode', 'state_abbr', 'address', 'cc_number', 'cc_expiry', 'cc_security_code',

```
'company_name', 'company_suffix', 'user_agent'],
dtype='object')
```

df.dtypes



	0
Unnamed: 0	int64
customer_id	object
full_name	object
gender	object
dob	object
age	int64
is_young_adult	object
is_senior	object
birth_country	object
passport_number	object
passport_expiry	object
phone_number	object
email	object
marital_status	object
has_children	object
number_dependents	int64
Gender	object
profile	object
locale	object
prefix	object
first_name	object
last_name	object
secondary_address	object
building_number	float64
street_name	object
street_suffix	object
city	object
postcode	float64
state_abbr	object
address	object
cc_number	float64
cc_expiry	object
cc_security_code	float64
company_name	object
company_suffix	object
user_agent	object



```
# <Student to fill this section>
dataset_insights = ""
The dataset contains 11,688 entries with 36 columns covering customer identity, demographics, and contact information.
Most of the key columns such as age, gender, marital status, and number of dependents are fully populated, indicating fairly complete baseli
However, approximately 18 columns are only populated for 1,688 entries, which are addresses, credit card numbers, and employers,
which are only available for a small subset of customers.
```

5/21/25, 9:28 PM36106-25AU-AT3-group<18>-<25741719>-eda-customer_details.ipynb - Colab

This suggests that the additional data is inconsistent and likely comes from different sources or phases of collection.
""

Do not modify this code
print_tile(size="h3", key='dataset_insights', value=dataset_insights)

dataset_insights

The dataset contains 11,688 entries with 36 columns covering customer identity, demographics, and contact information. Most of the key columns such as age, gender, marital status, and number of dependents are fully populated, indicating fairly complete baseline data. However, approximately 18 columns are only populated for 1,688 entries, which are addresses, credit card numbers, and

B.3 Explore Feature of Interest 1

You can add more cells in this section

```
# <Student to fill this section>  
df.describe()
```

	Unnamed: 0	age	number_dependents	building_number	postcode	cc_number	cc_security_code
count	11688.000000	11688.000000	11688.000000	1688.000000	1688.000000	1.688000e+03	1688.000000
mean	5843.500000	47.828970	0.694644	171.125000	3102.980450	4.149108e+17	884.476303
std	3374.179308	17.807161	1.168934	270.512982	1731.195095	1.300788e+18	1509.431934
min	0.000000	18.000000	0.000000	0.000000	202.000000	6.040156e+10	1.000000
25%	2921.750000	33.000000	0.000000	6.000000	2581.750000	1.800107e+14	277.000000
50%	5843.500000	47.000000	0.000000	39.000000	2689.000000	3.512907e+15	562.000000
75%	8765.250000	61.000000	1.000000	189.000000	2962.000000	4.663844e+15	821.000000

```
df.describe(exclude="number")
```

	customer_id	full_name	gender	dob	is_young_adult	is_senior	birth_country	passport_number	passport_expiry	phone_number
count	11688	11688	11688	11688	11688	11688	11688	11688	11688	1168
unique	8731	8118	3	7282	2	2	13	8252	1751	822
top	36b7369a-e3b5-4fba367-23da553a0b32	James Johnson	Male	1964-03-10	No	No	AU	A45935999	2026-06-20	+61.2.7054.025
freq	3	8	5645	8	9444	9298	10189	4	23	

4 rows x 29 columns

```
# <Student to fill this section>  
feature1_insights = ""  
From the numeric data, the average age of customers is around 47.8 years with an age range between 18 to 92 years, indicating a fairly wide age range. Most customers have no dependents (number_dependents median = 0), but some have up to 9 dependents. Additional data such as building_number, postcode, and credit card information (cc_number, cc_security_code) are only available for around 1,688 entries, indicating limitations in address and financial data coverage. The cc_number value has a very large range and standard deviation, indicating variation or the possibility of dummy/synthetic values.  
  
Meanwhile, for the categorical column, most customers are from Australia (birth_country top = AU), with the majority stating that they are not young (is_young_adult = No) and not elderly (is_senior = No), confirming that the majority are in the productive age group. There are many unique values in customer_id, full_name, and passport_number, indicating a high level of uniqueness of customer data. However, there are some minor repetitions, for example one customer_id appears 3 times, which could be a signal of potential duplication.  
""
```

```
# Do not modify this code
print_tile(size="h3", key='feature1_insights', value=feature1_insights)
```

feature1_insights

From the numeric data, the average age of customers is around 47.8 years with an age range between 18 to 92 years, indicating a fairly wide age distribution. Most customers have no dependents (number_dependents median = 0), but some have up to 9 dependents. Additional data such as building_number, postcode, and credit card information (cc_number, cc_security_code) are only available for around 1,688 customers, indicating limitations in address and financial data coverage. The cc_number value has a very large range and standard deviation, indicating variation or the possibility of dummy/synthetic values. Meanwhile, for the categorical column, most customers are from Australia (birth_country top = AU), with the majority stating that they are not young

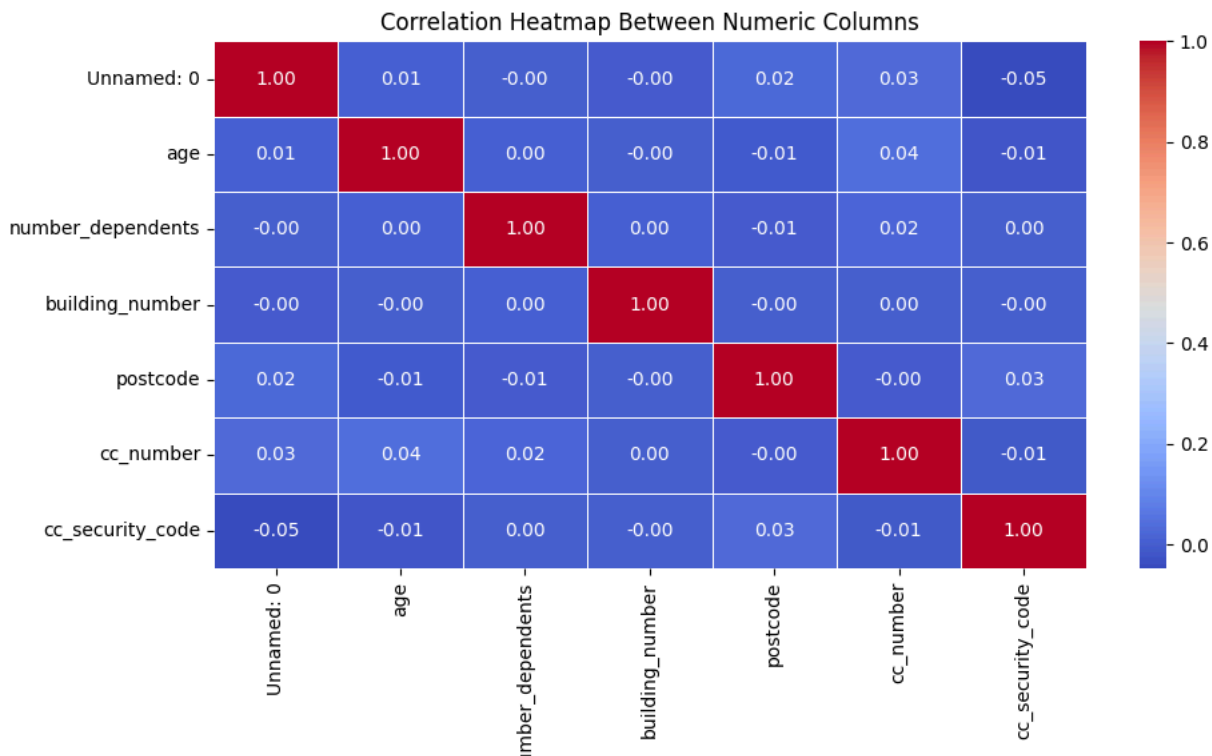
✓ B.4 Explore Feature of Interest 2

You can add more cells in this section

```
# <Student to fill this section>
# Select only numeric columns
numerical_df = df.select_dtypes(include=['int64', 'float64'])

# Calculate the correlation
corr_matrix = numerical_df.corr()

# Plot Heatmap
plt.figure(figsize=(10, 6))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f", linewidths=0.5)
plt.title("Correlation Heatmap Between Numeric Columns")
plt.tight_layout()
plt.show()
```



```
# Automatically identify categorical columns
categorical_cols = df.select_dtypes(include='object').columns.tolist()

# Filter columns by a reasonable number of uniques to plot (< 30 categories)
categorical_cols = [col for col in categorical_cols if df[col].nunique() < 30]

# Set the number of subplots
n_cols = 3
n_rows = (len(categorical_cols) + n_cols - 1) // n_cols
```

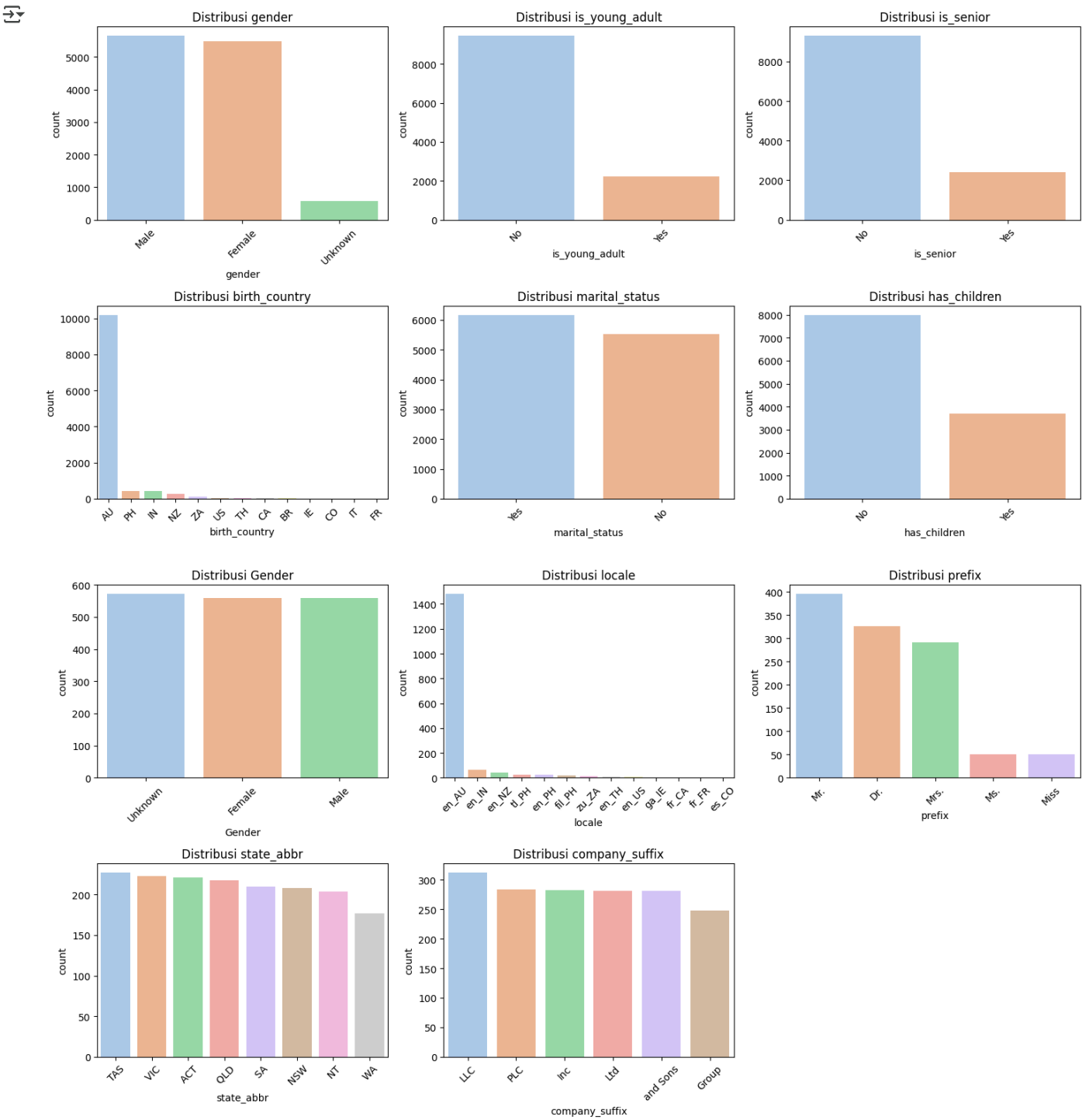


```
# Create a plot
fig, axes = plt.subplots(n_rows, n_cols, figsize=(n_cols*5, n_rows*4))
axes = axes.flatten()

# Plot each column
for i, col in enumerate(categorical_cols):
    sns.countplot(data=df, x=col, ax=axes[i], palette='pastel', order=df[col].value_counts().index)
    axes[i].set_title(f'Distribusi {col}')
    axes[i].tick_params(axis='x', rotation=45)

# Delete unused axes if any
for j in range(i+1, len(axes)):
    fig.delaxes(axes[j])

plt.tight_layout()
plt.show()
```



```
# Take all numeric columns
numerical_cols = df.select_dtypes(include=['int64', 'float64']).columns.tolist()

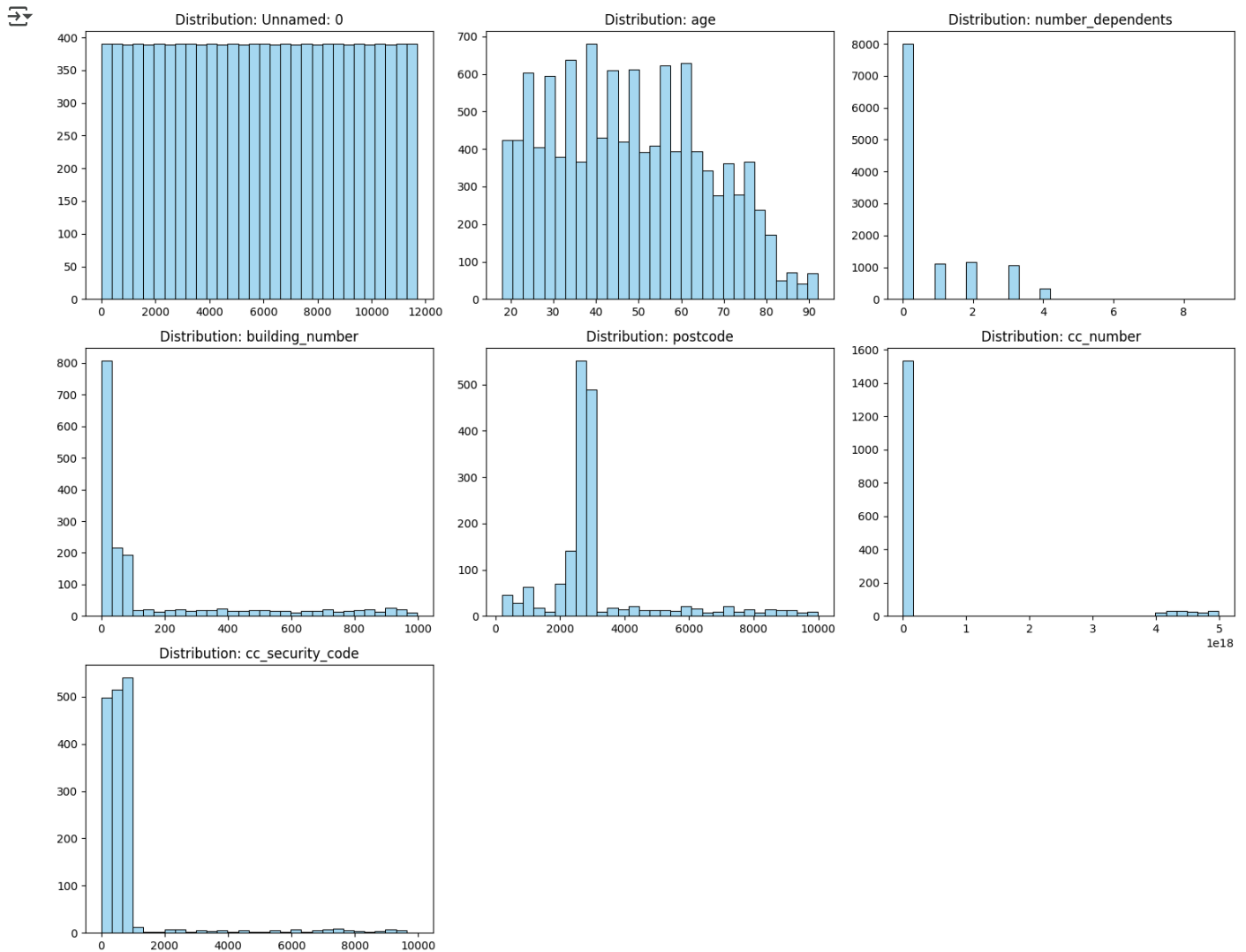
# Determine the number of grid rows and columns
n_cols = 3
n_rows = (len(numerical_cols) + n_cols - 1) // n_cols

# Make a plot
fig, axes = plt.subplots(n_rows, n_cols, figsize=(n_cols * 5, n_rows * 4))
axes = axes.flatten()

# Plot histogram
for i, col in enumerate(numerical_cols):
    sns.histplot(df[col], bins=30, kde=False, ax=axes[i], color='skyblue')
    axes[i].set_title(f'Distribution: {col}')
    axes[i].set_xlabel('')
    axes[i].set_ylabel('')

# Empty the remaining plot if the number of columns is not divisible by n_cols
for j in range(i + 1, len(axes)):
    fig.delaxes(axes[j])

plt.tight_layout()
plt.show()
```



<Student to fill this section>

feature_2_insights = ""

All correlation values are very close to zero, indicating that there is no strong linear relationship between the numeric features. The features age, number_dependents, building_number, postcode, cc_number, and cc_security_code are not correlated with each other, which means they can provide unique information in the model, but can also show high uncorrelation in the context of business and analysis.

Based on the visualization of the distribution of demographic data, the majority of individuals in this dataset are from Australia, with a nearly balanced gender proportion between men and women, and little unknown data.

Most individuals are neither young adults nor seniors, indicating a dominance of the middle adult age group.

Marital status shows a nearly balanced number between married and unmarried, but the majority do not have children.

This finding provides a picture of a relatively mature population, not too young or old, dominated by local residents, and most do not have

Then, it can be concluded that the majority of data uses the en_AU locale, confirming that the population is dominated by users from Austral. The gender distribution is quite balanced between men, women, and the unknown category.

The most commonly used prefix is "Mr.", followed by "Dr." and "Mrs.", indicating a predominance of male users and formal titles.

The distribution of state distribution in Australia is also quite even, with almost equal representation across states such as TAS, VIC, ACT. For business entities, the suffixes "LLC", "PLC", and "Inc" dominate, reflecting a diverse but fairly balanced form of corporate organizatio

The data displayed shows a diverse distribution. The age column shows a major concentration at the age of 30–60 years, reflecting the majority of respondents are in their productive age. Most have no dependents (`number_dependents = 0`). The distribution of `building_number` and `postcode` is very skewed to the left, indicating a predominance of low values, while `cc_number` appears extremely skewed with many small values, likely dummies. The `Unnamed: 0` column appears to be a unique ID that is even. Overall, the data is quite clean, although some numeric columns require special attention before being used for further analysis.

```
"""
```

```
# Do not modify this code
print_tile(size="h3", key='feature_2_insights', value=feature_2_insights)
```

```
feature_2_insights
```

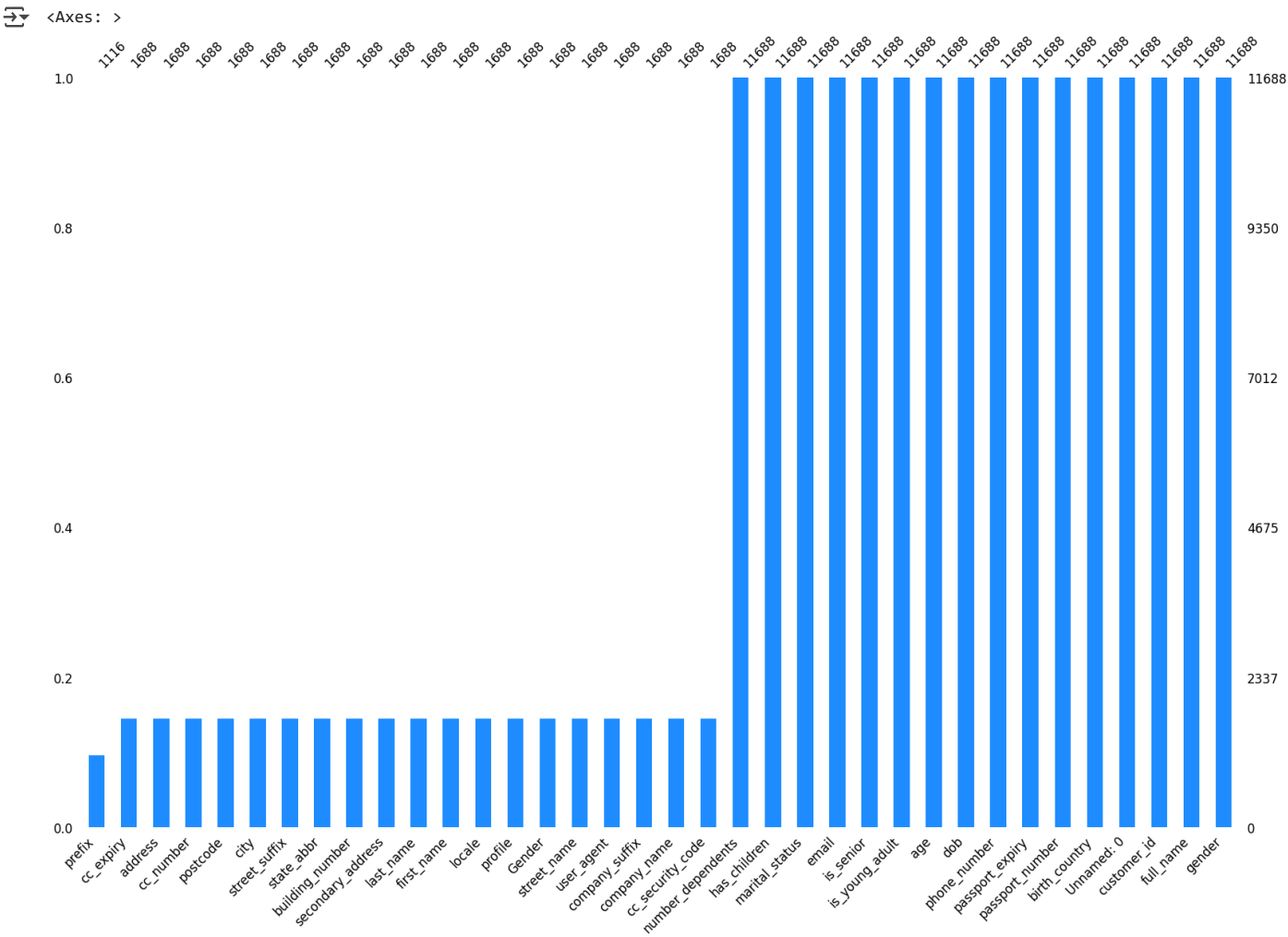
All correlation values are very close to zero, indicating that there is no strong linear relationship between the numeric features. The features `age`, `number_dependents`, `building_number`, `postcode`, `cc_number`, and `cc_security_code` are not correlated with each other, which means they can provide unique information in the model, but can also show high uncorrelation in the context of business and analysis. Based on the visualization of the distribution of demographic data, the majority of individuals in this dataset are from Australia, with a nearly balanced gender proportion between men and women, and little unknown data. Most individuals are neither young adults nor seniors, indicating a dominance of the middle adult age group. Marital status shows a nearly balanced number between married and unmarried, but the majority do not have children. This finding provides a picture of a relatively mature population, not too young or old, dominated by local residents, and most do not have children. Then, it can be concluded that the majority of data uses the `en_AU` locale, confirming that the population is dominated by users from Australia. The gender distribution is quite balanced between men, women, and the unknown category. The most commonly used prefix is "Mr.", followed by "Dr." and "Mrs.", indicating a predominance of male users and formal titles. The distribution of state distribution in Australia is also quite even, with almost equal representation across states such as TAS, VIC, ACT, to WA. For business entities, the suffixes "LLC", "PLC", and "Inc" dominate, reflecting a diverse but fairly balanced form of corporate organization. The data displayed shows a diverse

✓ B.5 Explore Feature of Interest n

You can add more cells in this section

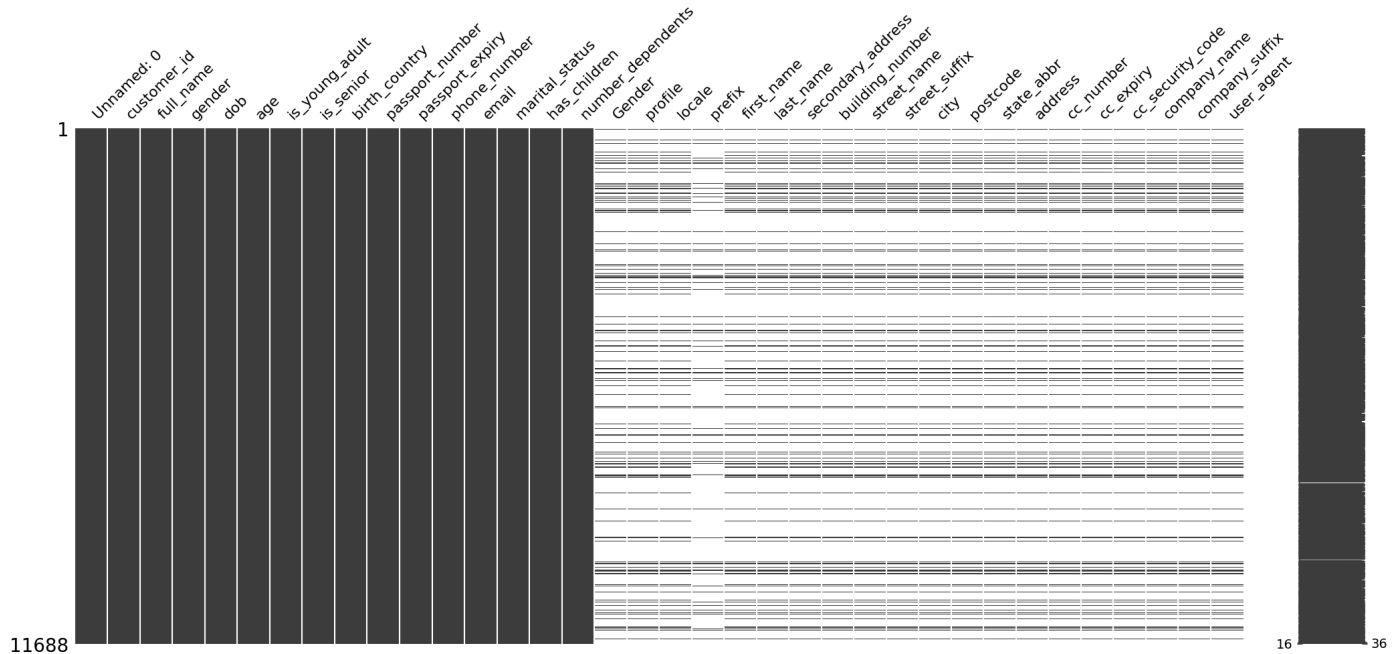
```
# <Student to fill this section>
missing_pct = df.isnull().mean()

msno.bar(df,
          color="dodgerblue",
          sort="ascending",
          figsize=(20, 13),
          fontsize=12)
```



msno.matrix(df)

↻ <Axes: >



<Student to fill this section>

```
feature_n_insights = ""
```

The missing value visualization above shows that the dataset consists of 11,688 entries and about 36 columns.

Each row represents one data entry, and each column represents a different feature.

The black color indicates that the data is available, while the white color indicates that there is missing data.

Likewise with the first visualization, the blue color indicates data availability.

It can be seen that the columns, such as customer_id, dob, age, and email have complete data.

Then, a number of columns in others, such as profile, prefix, secondary_address, cc_security_code have a significant number of missing value. This visualization is very helpful in the data cleaning process, because we can easily identify which columns need attention.

```
"""
```

Do not modify this code

```
print_tile(size="h3", key='feature_n_insights', value=feature_n_insights)
```

↻ feature_n_insights

The missing value visualization above shows that the dataset consists of 11,688 entries and about 36 columns. Each row represents one data entry, and each column represents a different feature. The black color indicates that the data is available, while the white color indicates that there is missing data. Likewise with the first visualization, the blue color indicates data availability. It can be seen that the columns, such as customer_id, dob, age, and email have complete data. Then, a number of columns in others, such as



✓ C. Data Cleaning

✓ C.1 Copy Dataset

Do not modify this code

```
df_clean = df.copy()
```

```
try:
```

```
    df_clean = df.copy()
```

```
except Exception as e:
```

```
    print(e)
```

✓ C.2 Fixing "<Duplicates>"

Provide some explanations on why you believe it is important to fix this issue and its impacts

You can add more cells in this section

```
# <Student to fill this section>
df_clean = df_clean.drop(columns=['Unnamed: 0'], errors='ignore')
```

```
print("Duplicate at the df:", df_clean.duplicated().sum())
```

```
↗ Duplicate at the df: 2957
```

```
df_clean = df_clean.drop_duplicates()
```

```
df_clean.info()
```

```
↗ <class 'pandas.core.frame.DataFrame'>
Index: 8731 entries, 0 to 11686
Data columns (total 35 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   customer_id           8731 non-null   object
1   full_name             8731 non-null   object
2   gender                8731 non-null   object
3   dob                  8731 non-null   object
4   age                   8731 non-null   int64
5   is_young_adult        8731 non-null   object
6   is_senior             8731 non-null   object
7   birth_country         8731 non-null   object
8   passport_number       8731 non-null   object
9   passport_expiry       8731 non-null   object
10  phone_number          8731 non-null   object
11  email                 8731 non-null   object
12  marital_status        8731 non-null   object
13  has_children          8731 non-null   object
14  number_dependents     8731 non-null   int64
15  Gender                1688 non-null   object
16  profile               1688 non-null   object
17  locale                1688 non-null   object
18  prefix                1116 non-null   object
19  first_name            1688 non-null   object
20  last_name             1688 non-null   object
21  secondary_address     1688 non-null   object
22  building_number       1688 non-null   float64
23  street_name           1688 non-null   object
24  street_suffix         1688 non-null   object
25  city                  1688 non-null   object
26  postcode              1688 non-null   float64
27  state_abbr            1688 non-null   object
28  address               1688 non-null   object
29  cc_number             1688 non-null   float64
30  cc_expiry             1688 non-null   object
31  cc_security_code      1688 non-null   float64
32  company_name          1688 non-null   object
33  company_suffix        1688 non-null   object
34  user_agent            1688 non-null   object
dtypes: float64(4), int64(2), object(29)
memory usage: 2.4+ MB
```

```
# <Student to fill this section>
data_cleaning_1_explanations = """
The Unnamed: 0 column is removed because it is only an index from the CSV file and does not carry any important information.
After that, duplicate data is removed to ensure each row represents a unique entity and prevent bias or double counting in the analysis.
"""
```

```
# Do not modify this code
print_tile(size="h3", key='data_cleaning_1_explanations', value=data_cleaning_1_explanations)
```

```
↗ data_cleaning_1_explanations
```

The Unnamed: 0 column is removed because it is only an index from the CSV file and does not carry any important information. After

✓ C.3 Fixing "<Missing Value>"

Provide some explanations on why you believe it is important to fix this issue and its impacts

You can add more cells in this section

```
# <Student to fill this section>
df_clean.isnull().sum()
```



	0
customer_id	0
full_name	0
gender	0
dob	0
age	0
is_young_adult	0
is_senior	0
birth_country	0
passport_number	0
passport_expiry	0
phone_number	0
email	0
marital_status	0
has_children	0
number_dependents	0
Gender	7043
profile	7043
locale	7043
prefix	7615
first_name	7043
last_name	7043
secondary_address	7043
building_number	7043
street_name	7043
street_suffix	7043
city	7043
postcode	7043
state_abbr	7043
address	7043
cc_number	7043
cc_expiry	7043
cc_security_code	7043
company_name	7043
company_suffix	7043
user_agent	7043

```
cols_to_drop = ['Gender', 'profile', 'locale', 'prefix', 'first_name', 'last_name',
                'secondary_address', 'building_number', 'street_name', 'street_suffix',
                'city', 'postcode', 'state_abbr', 'address', 'cc_number', 'cc_expiry',
                'cc_security_code', 'company_name', 'company_suffix', 'user_agent']
```

```

        'phone_number', 'email']

df_clean = df_clean.drop(columns=cols_to_drop)

```

```
df_clean.info()
```

```

<class 'pandas.core.frame.DataFrame'>
Index: 8731 entries, 0 to 11686
Data columns (total 15 columns):
 #   Column                Non-Null Count  Dtype
---  ---
 0   customer_id           8731 non-null   object
 1   full_name              8731 non-null   object
 2   gender                 8731 non-null   object
 3   dob                    8731 non-null   object
 4   age                    8731 non-null   int64
 5   is_young_adult         8731 non-null   object
 6   is_senior              8731 non-null   object
 7   birth_country          8731 non-null   object
 8   passport_number        8731 non-null   object
 9   passport_expiry        8731 non-null   object
10   phone_number           8731 non-null   object
11   email                  8731 non-null   object
12   marital_status         8731 non-null   object
13   has_children           8731 non-null   object
14   number_dependents      8731 non-null   int64
dtypes: int64(2), object(13)
memory usage: 1.1+ MB

```

```
# <Student to fill this section>
```

```
data_cleaning_2_explanations = ""
```

In dealing with missing values, I took an approach with a relevance evaluation on the domain.

The result was that the variables gender, profile, locale, first_name, last_name, address, secondary_address, phone_number, and email were removed because there were too many missing, biased, and personal information that was not relevant to the project objectives or target variables. Then, the variables prefix, building_number, street_name, street_suffix, city, postcode, and state_abbr were also removed because they had low values for the model and were difficult to impute.

After that, the variables cc_number, cc_expiry, and cc_security_code were also removed because they were too sensitive for modeling and not the variables company_name, company_suffix, and user_agent were removed because they had many missing values, were not directly relevant to revenue, and their information extraction required additional effort that was likely not comparable to the added value in revenue prediction.

```
"""
```

```
# Do not modify this code
```

```
print_tile(size="h3", key='data_cleaning_2_explanations', value=data_cleaning_2_explanations)
```

```
data_cleaning_2_explanations
```

In dealing with missing values, I took an approach with a relevance evaluation on the domain. The result was that the variables gender, profile, locale, first_name, last_name, address, secondary_address, phone_number, and email were removed because there were too many missing, biased, and personal information that was not relevant to the project objectives or target variables. Then, the variables prefix, building_number, street_name, street_suffix, city, postcode, and state_abbr were also removed because they had low values for the model and were difficult to impute. After that, the variables cc_number, cc_expiry, and cc_security_code were also

✓ C.4 Fixing "<Data Formatting>"

Provide some explanations on why you believe it is important to fix this issue and its impacts

You can add more cells in this section

```
# <Student to fill this section>
```

```
df_clean['dob'] = pd.to_datetime(df_clean['dob'], errors='coerce')
```

```
df_clean['passport_expiry'] = pd.to_datetime(df_clean['passport_expiry'], errors='coerce')
```

```
categorical_cols = ['gender', 'is_young_adult', 'is_senior', 'birth_country',
                    'marital_status', 'has_children']
```

```
df_clean[categorical_cols] = df_clean[categorical_cols].astype('category')
```

```
df_clean.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 8731 entries, 0 to 11686
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  -
0   customer_id            8731 non-null   object
1   full_name              8731 non-null   object
2   gender                 8731 non-null   category
3   dob                   8731 non-null   datetime64[ns]
4   age                   8731 non-null   int64
5   is_young_adult         8731 non-null   category
6   is_senior              8731 non-null   category
7   birth_country          8731 non-null   category
8   passport_number        8731 non-null   object
9   passport_expiry        8731 non-null   datetime64[ns]
10  marital_status         8731 non-null   category
11  has_children           8731 non-null   category
12  number_dependents      8731 non-null   int64
dtypes: category(6), datetime64[ns](2), int64(2), object(3)
memory usage: 598.1+ KB
```

df_clean.head()

	customer_id	full_name	gender	dob	age	is_young_adult	is_senior	birth_country	passport_number	passport_expiry	marital_status
0	9acb3a1e-a040-4280-887b-ef33381c7a16	Alexa Chung	Female	1974-10-10	50	No	No	AU	131053534	2029-11-21	Yes
1	45360147-4c3e-4f3d-8d8d-a6aa8f0bb47f	Mrs. Megan	Female	1981-09-27	43	No	No	AU	379521345	2028-04-11	Yes
2	8345bdc5-75fd-41da-8ce5-7828e5a27666	Amy Rose	Female	1989-09-16	35	No	No	PH	794022868	2028-12-13	Yes
3	0ec226b1-22e6-4b98-919f-	Ryan Peterson	Male	2001-02-03	24	Yes	No	AU	M21113240	2027-05-23	No

Next steps: [Generate code with df_clean](#) [View recommended plots](#) [New interactive sheet](#)

```
# <Student to fill this section>
data_cleaning_3_explanations = ""
I converted the data type in column dob and passport expiry to date format, and changed several text type columns to categories as a form of ""

# Do not modify this code
print_tile(size="h3", key='data_cleaning_3_explanations', value=data_cleaning_3_explanations)

data_cleaning_3_explanations
```

▼ D. Feature Engineering

▼ D.1 Copy Datasets

```
# Do not modify this code
try:
    df_eng = df_clean.copy()
except Exception as e:
    print(e)
```

▼ D.2 New Feature "<Data Enrichment>"

Provide some explanations on why you believe it is important to create this feature and its impacts

```
# <Student to fill this section>
df1 = df1.drop(columns=['Unnamed: 0'], errors='ignore')
print("Duplicate in the df1:", df1.duplicated().sum())
```

↗ Duplicate in the df1: 4082

```
df1 = df1.drop_duplicates()
df1.info()
```

↗

```
<class 'pandas.core.frame.DataFrame'>
Index: 7043 entries, 0 to 11124
Data columns (total 9 columns):
#   Column                                Non-Null Count  Dtype
---  ---                                -
0   customer_id                          7043 non-null   object
1   avg_monthly_international_charges    7043 non-null   float64
2   avg_monthly_download_in_gb          7043 non-null   int64
3   monthly_charge                       7043 non-null   float64
4   total_charges                        6933 non-null   float64
5   total_refunds                        6823 non-null   float64
6   total_extra_data_charges             6933 non-null   float64
7   total_international_charges          6209 non-null   float64
8   total_revenue                        5816 non-null   float64
dtypes: float64(7), int64(1), object(1)
memory usage: 550.2+ KB
```

```
df_eng.info()
```

↗

```
<class 'pandas.core.frame.DataFrame'>
Index: 8731 entries, 0 to 11686
Data columns (total 13 columns):
#   Column                                Non-Null Count  Dtype
---  ---                                -
0   customer_id                          8731 non-null   object
1   full_name                            8731 non-null   object
2   gender                               8731 non-null   category
3   dob                                  8731 non-null   datetime64[ns]
4   age                                  8731 non-null   int64
5   is_young_adult                       8731 non-null   category
6   is_senior                            8731 non-null   category
7   birth_country                       8731 non-null   category
8   passport_number                     8731 non-null   object
9   passport_expiry                     8731 non-null   datetime64[ns]
10  marital_status                       8731 non-null   category
11  has_children                         8731 non-null   category
12  number_dependents                   8731 non-null   int64
dtypes: category(6), datetime64[ns](2), int64(2), object(3)
memory usage: 598.1+ KB
```

```
df_eng = df_eng.merge(df1, on='customer_id', how='inner')
```

```
df_eng.info()
```

↗

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
#   Column                                Non-Null Count  Dtype
---  ---                                -
0   customer_id                          7043 non-null   object
1   full_name                            7043 non-null   object
2   gender                               7043 non-null   category
3   dob                                  7043 non-null   datetime64[ns]
4   age                                  7043 non-null   int64
5   is_young_adult                       7043 non-null   category
6   is_senior                            7043 non-null   category
7   birth_country                       7043 non-null   category
8   passport_number                     7043 non-null   object
9   passport_expiry                     7043 non-null   datetime64[ns]
10  marital_status                       7043 non-null   category
11  has_children                         7043 non-null   category
12  number_dependents                   7043 non-null   int64
13  avg_monthly_international_charges    7043 non-null   float64
14  avg_monthly_download_in_gb          7043 non-null   int64
15  monthly_charge                       7043 non-null   float64
16  total_charges                        6933 non-null   float64
17  total_refunds                        6823 non-null   float64
18  total_extra_data_charges             6933 non-null   float64
```

```

19 total_international_charges      6209 non-null    float64
20 total_revenue                     5816 non-null    float64
dtypes: category(6), datetime64[ns](2), float64(7), int64(3), object(3)
memory usage: 868.0+ KB

```

```
print("Duplicate in the df_eng:", df_eng.duplicated().sum())
```

```
↗ Duplicate in the df_eng: 0
```

```
# <Student to fill this section>
```

```
feature_engineering_1_explanations = """
```

In this project, the problem to be solved is to help the Finance Team adjust the budget by predicting next month's revenue.

It should be noted that the revenue variable is not available in the customer detail data.

Therefore, a merger between customer detail data and customer usage is carried out based on customer_id.

After the merger process between df_eng (customer data) and df1 (usage and revenue data), 7,043 rows of customer data were obtained that had both in terms of demographics and service usage behavior.

```
"""
```

```
# Do not modify this code
```

```
print_tile(size="h3", key='feature_engineering_1_explanations', value=feature_engineering_1_explanations)
```

```
↗ feature_engineering_1_explanations
```

In this project, the problem to be solved is to help the Finance Team adjust the budget by predicting next month's revenue. It should be noted that the revenue variable is not available in the customer detail data. Therefore, a merger between customer detail data and customer usage is carried out based on customer_id. After the merger process between df_eng (customer data) and df1 (usage and

✓ D.3 New Feature "<refund_ratio and extra_data_ratio>"

Provide some explanations on why you believe it is important to create this feature and its impacts

```
# <Student to fill this section>
```

```
df_eng["refund_ratio"] = df_eng["total_refunds"] / df_eng["total_charges"].replace(0, np.nan)
```

```
df_eng["extra_data_ratio"] = df_eng["total_extra_data_charges"] / df_eng["total_charges"].replace(0, np.nan)
```

```
df_eng.info()
```

```

↗ <class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 23 columns):
 #   Column                                Non-Null Count  Dtype
---  -
 0   customer_id                          7043 non-null   object
 1   full_name                            7043 non-null   object
 2   gender                               7043 non-null   category
 3   dob                                  7043 non-null   datetime64[ns]
 4   age                                  7043 non-null   int64
 5   is_young_adult                       7043 non-null   category
 6   is_senior                            7043 non-null   category
 7   birth_country                        7043 non-null   category
 8   passport_number                     7043 non-null   object
 9   passport_expiry                     7043 non-null   datetime64[ns]
10  marital_status                       7043 non-null   category
11  has_children                         7043 non-null   category
12  number_dependents                    7043 non-null   int64
13  avg_monthly_international_charges    7043 non-null   float64
14  avg_monthly_download_in_gb           7043 non-null   int64
15  monthly_charge                       7043 non-null   float64
16  total_charges                        6933 non-null   float64
17  total_refunds                        6823 non-null   float64
18  total_extra_data_charges             6933 non-null   float64
19  total_international_charges          6209 non-null   float64
20  total_revenue                        5816 non-null   float64
21  refund_ratio                         6720 non-null   float64
22  extra_data_ratio                     6824 non-null   float64
dtypes: category(6), datetime64[ns](2), float64(9), int64(3), object(3)
memory usage: 978.1+ KB

```

```
# <Student to fill this section>
```

```
feature_engineering_2_explanations = """
```

In the context of revenue prediction, creating financial ratio features such as refund_ratio and extra_data_ratio is more relevant, because it is directly related to money, captures customer efficiency or inefficiency, and increases signals to the model.